

Stochastic Modeling of Funding Rate Dynamics with Jumps*

Shreyash Kharat

Indian Statistical Institute Kolkata

April 2025

Abstract

This paper analyzes the behavior of funding rates in the BTCUSDT perpetual futures market through constructing and comparing mean-reverting stochastic models with and without jumps. With minute-level funding rate data from Binance for an eight-day subsample (17–24 December 2024), we begin with a detailed descriptive analysis: stationarity testing using the Augmented Dickey–Fuller test, serial dependence investigation using ACF and PACF, and testing distributional properties such as skewness, kurtosis, and log-normality. We propose a two-stage transformation to normalize the series. Then, we estimate an Ornstein–Uhlenbeck (OU) mean-reversion model by maximizing its transition density likelihood through a hybrid Nelder–Mead and grid-search procedure, cross-validating on simulated data and predicting out-of-sample (25 December 2024) with one-step-ahead prediction and Monte Carlo simulation. Finding large unexplained variance, we add Gaussian-compounded jumps to the model. We estimate the parameters of the extended model using a Metropolis–Hastings MCMC algorithm but face computational bottlenecks. To this end, we propose a fast, Euler–Maruyama-based jump-detection test that better controls Type I error than Lee–Mykland. By isolating diffusion and jump components, we re-estimate parameters by $AR(1)$ for OU and empirical moments for jumps. We summarize by describing applications to high-frequency trading algorithms and a rolling-window framework for dynamic real-time parameter update.

1 Introduction

Bitcoin (BTC) is the first and most widely adopted cryptocurrency, introduced by an anonymous entity under Satoshi Nakamoto’s pseudonym in 2008. It operates on a decentralized ledger called the blockchain, which ensures transparency, security, and immutability through cryptographic mechanisms. The BTC market has experienced rapid growth and significant volatility, driven by institutional adoption, retail speculation, regulatory developments, and macroeconomic factors. The

*I gratefully acknowledge the guidance of Prof. Diganta Mukherjee (SOSU, ISI Kolkata) and Prof. Indranil SenGupta (Hunter College, CUNY) during this work.

blockchain's decentralized nature ensures that no single entity controls the ledger, making Bitcoin a trustless and censorship-resistant system.

The BTC market consists of several key trading instruments, including spot trading, futures contracts, options, and perpetual futures. Unlike traditional financial assets, BTC exhibits unique statistical properties such as heavy-tailed distributions, volatility clustering, and potential inefficiencies, making it an interesting subject for stochastic modeling. The dynamics of BTC prices can be captured using various stochastic processes, including Geometric Brownian Motion (GBM), Mean-Reverting Processes, and Jump-Diffusion models, depending on the properties being analyzed.

In the digital assets market, perpetual futures are one of the most widely traded and most liquid instruments with more than \$ 60 trillion traded starting in 2020. Perpetual futures are futures contracts without expiration. They depend on **funding rate** mechanism to keep prices of contract close to that of underlying assets.

Whenever the price of the contract is greater than the price of the spot, we say that the funding rate is positive and in this case, the longs pay the funding price to the shorts, and if the contract price is less than that of spot, we say that the funding rate is negative and in this case the shorts pay funding price to the longs. Funding price is the product of funding rate, price of the contract at settlement, and number of the contract held. The funding rate settlement is done at 0 UTC, 8 UTC, and 16 UTC i.e. three times a day.

The goal of funding rates is to keep the prices of the spot BTCUSDT and its perpetual future contract close. So, when the prices of the spot and contract diverge, the exchange incentivizes traders to take positions so that the prices of the spot and contract come close. For example, let us say that the current time is 06:00 UTC and the spot is trading at \$ 95000 and the contract is trading at \$ 95500. Since the contract price is greater than the spot price the exchange should incentivize trader to take a short position, so traders who take the short position can take home the funding rate settlement at 08:00 UTC given that the funding rates are still positive at 08:00 UTC. If at 06:00 UTC the contract was trading at \$ 94000, the exchanges would have incentivized long traders, in which case the trader who took long positions would be rewarded with the settlement at 08:00 UTC given the funding are negative at 08:00 UTC.

Funding rate calculation are specific to the exchanges i.e., the funding rate calculation for Binance, Bybit will differ. Since, Binance has the most liquidity for perpetual futures, we will calculate funding rate as Binance does. Following is the formula for funding rate:

$$\text{Funding Rate (F)} = \text{Weighted Average Premium Index (P) over last 8 hours} + \text{clamp}(\text{interest rate} - \text{Premium Index (P)}, 0.05\%, -0.05\%),$$

where the interest rate is the interest on borrowing the settlement stable-coin - USDT, the clamp is defined as:

$$\text{clamp}(x, t, s) = \max(\min(x, t), s), \quad t < s.$$

Hence, to calculate funding rates we require the data for the Premium Index at a desired

granularity. We can obtain the data for the premium index of BTCUSDT from Binance APIs. Here we have retrieve minute-wise data of the premium index of BTCUSDT from 01-01-2024 to 31-12-2024. Due to the large size of data, we will consider an 8-day period for further investigation which is 17-12-2024 to 24-12-2024. The reason for choosing this period is that we have enough historical data in the past if required.

Mathematically funding rates are calculated as,

$$f_t = \max(\min(r - p_{t-1}, -0.0005), 0.0005) + \frac{2}{480 \cdot 481} \sum_{i=1}^{480} (480 - i) \cdot p_{t-i}, \quad (1)$$

where f_t, p_t are funding rates and premium index respectively.

We begin with the Descriptive Analysis part reports stationarity testing (ADF), autocorrelation diagnostics (ACF/PACF), and distributional checks, leading to a log-normalizing transformation. Then we derive the Ornstein–Uhlenbeck Model, its transition density, maximum likelihood parameter estimation (Nelder–Mead + grid search), and one-step prediction and Monte Carlo simulation-based forecasting evaluation. Seeing model insufficiency, we introduce the OU with Gaussian-Compounded Jumps extension, reporting its SDE, Metropolis–Hastings estimation, and forecasting performance. Averting MCMC’s computational expense, we present an Efficient Jump Separation approach: we summarize the Lee–Mykland test, deduce an Euler–Maruyama-based equivalent with better error control, and apply it to split funding rates into diffusion and jump parts. We then estimate diffusion parameters by $AR(1)$ and jump statistics empirically, with major gains in forecast accuracy. We finish with Discussion on algorithmic trading applications and set out Future Directions, such as rolling-window updating and integration of exogenous signals.

2 Descriptive Study

We now proceed to descriptive study of the funding rates times series f_t . In this section we will be performing preliminary analysis on the time series to see patters, trend, stationarity, and distributional properties. These empirical findings will serve as the foundation for our subsequent analysis, informing both our model selection criteria and estimation methodology. Understanding these fundamental properties is essential for developing robust models that accurately capture the dynamics of funding rates in cryptocurrency markets. [Table 1](#) contains some preliminary statistics of the time series f_t , [Figure 1](#) is the plot of time series f_t over the 8-day period while [Figure 2](#) is the closer look at the time series f_t over 17-12-2024.

2.1 Stationarity

For testing Stationarity we will be using an Augmented Dickey Fuller Test (ADF Test). The ADF test is based on the following regression equation:

$$\Delta y_t = \alpha + \beta t + \gamma y_{t-1} + \sum_{i=1}^p \delta_i \Delta y_{t-i} + \varepsilon_t, \quad (2)$$

where: y_t is the time series value at time t , Δy_t is the first difference of y_t (i.e., $y_t - y_{t-1}$), α is the constant term (drift), βt is the time trend term, γ is the coefficient of interest for the unit root test, p is the lag order, δ_i are the coefficients of the lagged differences, ε_t is the error term.

The test examines the following hypotheses: $H_0 : \gamma = 0$ (Unit root exists, series is non-stationary) vs $H_1 : \gamma < 0$ (No unit root, series is stationary).

The test statistic is calculated as:

$$\text{ADF} = \frac{\hat{\gamma}}{\text{SE}(\hat{\gamma})} \quad (3)$$

where: $\hat{\gamma}$ is the estimated value of γ , $\text{SE}(\hat{\gamma})$ is the standard error of $\hat{\gamma}$.

For practical purposes, we have chosen following parameters for the test: the maximum lag considered is $12(n/100)^{1/4}$, and $n = 11520$, since we want to test for stationarity around a constant term, we do not include the time trend term (βt) in calculations.

Following were the results of the ADF test on observed funding rates during 17-12-2024 to 24-01-2024: $\text{ADF} = -8.401317$, 5% Critical value = -2.862, and 1% Critical Value = -3.431. Since obtained statistic is less than 1% Critical value, we have enough evidence to reject the Null Hypothesis of non-stationarity at a 99% level of confidence. So we conclude that the observed funding rates are stationary.

2.2 Autocorrelation Function and Partial Autocorrelation Function

Time series analysis employs two fundamental tools for understanding temporal dependencies: the Autocorrelation Function (ACF) and the Partial Autocorrelation Function (PACF). These functions provide crucial insights into the underlying structure of time series data and facilitate appropriate model selection.

For a time series $\{Y_t\}$, the ACF at lag k , denoted as ρ_k , is defined as: $\rho_k = \frac{\gamma_k}{\gamma_0}$ where γ_k is the autocovariance at lag k is $\gamma_k = E[(Y_t - \mu)(Y_{t+k} - \mu)]$ and γ_0 is the variance of the time series $\gamma_0 = E[(Y_t - \mu)^2]$.

The Partial Autocorrelation Function (PACF) extends this analysis by isolating direct relationships between observations at specific lags. The PACF at lag k , denoted as ϕ_{kk} , represents the correlation between Y_t and Y_{t+k} after removing the effects of intermediate observations. It can be computed using the following recursive relationship (Durbin-Levinson algorithm):

$$\phi_{kk} = \frac{\rho_k - \sum_{j=1}^{k-1} \phi_{k-1,j} \rho_{k-j}}{1 - \sum_{j=1}^{k-1} \phi_{k-1,j} \rho_j} \quad (4)$$

where:

$$\phi_{k,j} = \phi_{k-1,j} - \phi_{kk} \phi_{k-1,k-j} \quad \text{for } j = 1, 2, \dots, k-1 \quad (5)$$

In practice, sample versions of these functions are used, where expectations are replaced by sample means and variances:

Sample ACF:

$$\hat{r}_k = \frac{\hat{c}_k}{\hat{c}_0} \quad (6)$$

where:

$$\hat{c}_k = \frac{1}{n} \sum_{t=1}^{n-k} (y_t - \bar{y})(y_{t+k} - \bar{y}) \quad (7)$$

$$\hat{c}_0 = \frac{1}{n} \sum_{t=1}^n (y_t - \bar{y})^2 \quad (8)$$

The sample PACF is typically computed using least squares regression applied to the sample autocorrelation.

Figure 3 shows the ACF and PACF plots of the time series f_t . Note that the decaying nature of ACF above significance level is expected, due to the weighted average in the formula for funding rates. The PACF plot also demonstrates decaying PACF at insignificant levels after lag 2.

2.3 Distributional Analysis

In this part, we will study the distributional properties of the time series. First we will look at histograms and then proceed to transform the data for Log-normality and then to normal. Figure 4 is the histogram of the series f_t . We can see that the distribution is left-skewed implying the non-normality of the data. We can also see that the majority of density is concentrated around mode of f_t , also in Figure 1 we can see a lot of jumps to negative values, which shows up in the Histogram. So, we can follow two approaches, first either do some transformations and work with the complete data or second, separate the jumps and study the the series in two parts.

We will use a Kolmogorov-Smirnoff statistic to check if the data comes from a specific density. We will rather use the statistic as a measure of deviation from the best-fit density (Likelihood wise). The KS Density Deviation is defined as:

$$D_n = \sup_x |F_n(x) - F(x)|, \quad (9)$$

where $F_n(\cdot)$ is the empirical CDF of the data.

Since there's a left skew in the data, we transformed the data as $f'_t = 4 - f_t$, to make it positive and right-skewed. Then we tested whether the transformed data comes from a log-normal distribution, unfortunately the test rejected the null hypothesis of log-normal distribution. Figure 5 shows the best fit log-normal density on the data, we can see that the density near the mode has the highest deviation, so we will be transforming the funding rates series again to reduce the deviation at mode. Following is the transformation:

$$x_t = \begin{cases} m + |f'_t - m|^{\alpha_1}, & \text{if } f'_t \geq m \\ m - \beta \cdot |f'_t - m|^{\alpha_2}, & \text{if } f'_t < m \end{cases}, \quad (10)$$

where m is the mode of the Kernel Density Estimate of f'_t , and $\alpha_1, \alpha_2, \beta$ are parameters of transformation. Note that we can successfully reconstruct the series f'_t from x_t and hence also reconstruct the series f_t . the optimal parameters for transformation to get the best fitted log-normal distribution are $\alpha_1 = 0.805, \alpha_2 = 0.9, \beta = 3.1$. The obtained best fitted Log-Normal distribution has

parameters $s = 0.0314$, $\mu = 3.9770$, $\sigma = 0.0230$, in the density below. [Figure 6](#) shows the Best Fit density on x_t .

$$f(x) = \frac{1}{y\sigma\sqrt{2\pi}} \exp\left(-\frac{(\ln(y))^2}{s^2}\right), y = \frac{x - \mu}{\sigma}. \quad (11)$$

Results

We now compare a few statistics of the series f'_t, x_t to see if applying the transformation has indeed improved the fit of Log-normal distribution on the data. [Table 2](#) shows comparison of f'_t, x_t over statistics like skewness, kurtosis and Kolmogorov Smirnov Density Deviation from the best fit Log Normal density. We can straightaway see the improvement from f'_t to x_t . Both skewness, excess kurtosis of x_t is closer to the skewness of its best fit density than that of f'_t . We can also significant improve in the KS density deviation statistic. Based on the above evidence, we can conclude that the transformation has made an improvement in the fitting of the Log-Normal Density.

In conclusion, we know that f_t is stationary, has no seasonality upto 60 minutes, and x_t , which is a transformed version of f_t , has Log-normal distribution. Let $y_t = \log((x_t - \mu)/\sigma)$, implies y_t is normally distributed, [Figure 7](#) shows the time indexed plot of the series y_t , [Figure 8](#) shows the histogram and KDE of y_t which is very close to a normal distribution.

3 Ornstein-Uhlenbeck Process

Now that we have performed an in-detailed descriptive study of the funding rates series f_t and transformed it successfully into a series x_t for which we can proceed with Normality assumptions, we proceed to the modeling part of the analysis. In Section 2.1: [Stationarity](#) we concluded that the series f_t is stationary, similarly using an ADF test we can say that y_t is also stationary. Further since y_t is stationary, if we are going to model it using a stochastic process, the process shouldn't have a drift term i.e., the process shouldn't increase or decrease with time. Hence, we should begin with Ornstein-Uhlenbeck Mean Reversion Process or shortly OU Process.

3.1 Theory

The Ornstein-Uhlenbeck SDE is defined as:

$$dx_t = \theta(\mu - x_t)dt + \sigma dB_t, \quad (12)$$

where $\theta > 0$ is the speed of mean-reversion, μ is the long-term mean, $\sigma > 0$ is the volatility and B_t is the standard Brownian Motion or the Wiener Process.

To get a solution of OU Process we will first solve the SDE: $dr_t = -\theta r_t dt + \sigma dB_t$. We will use the Ito's formula along with $f(r, t) = r \exp(\theta t)$. This yields:

$$\frac{\partial f}{\partial t} = f\theta, \frac{\partial f}{\partial r} = \exp(\theta t), \frac{\partial^2 f}{\partial r^2} = 0, \quad (13)$$

and $dt dr = dt(-\lambda dt + \sigma dB_t) = 0$. Substituting the above in the Ito's formula yields,

$$df = \sigma \exp(\theta t) dB_t, \quad (14)$$

$$\implies f_t = f_0 + \sigma \int_{s=0}^t \exp(\theta s) dB_s, \quad (15)$$

$$\implies r_t = r_0 \exp(-\theta t) + \exp(-\theta t) \sigma \int_{s=0}^t \exp(\theta s) dB_s. \quad (16)$$

The above stochastic integral is Normally distributed since its integrand is a non-random function, implies r_t is also normally distributed. Our next task is find the expectation and variance of r_t . Since, the stochastic integral has expectation zero,

$$E(r_t) = \exp(-\theta t), \quad (17)$$

and for variance,

$$\text{Var}(r_t) = E[(r_t - E(r_t))^2] \quad (18)$$

$$= E\left[\left(\sigma \exp(-\theta t) \int_{s=0}^t \exp(\theta s) dB_s\right)^2\right] \quad (19)$$

$$= (\sigma \exp(-\theta t))^2 \int_{s=0}^t E[\exp(\theta s)^2] ds, \quad (20)$$

since $\exp(\theta s)^2$ is non-random, which implies:

$$\text{Var}(r_t) = \frac{\sigma^2}{2\theta} [1 - \exp(-2\theta t)]. \quad (21)$$

Then, $r_t \sim N\left(\exp(-\theta t), \frac{\sigma^2}{2\theta} [1 - \exp(-2\theta t)]\right)$. If we make the transformation $r_t = x_t - \mu$, we can conclude that

$$x_t \sim N\left(x_0 \exp(-\theta t) + \mu [1 - \exp(-\theta t)], \frac{\sigma^2}{2\theta} [1 - \exp(-2\theta t)]\right). \quad (22)$$

3.2 Maximum Likelihood Estimation

We now move forward to estimating the parameters of the mean reversion process. Given the distribution in Equation 22 we can write a Likelihood function for the parameters θ, μ, σ . First we will be simulating a sample path of the mean reversion process using the discretization obtained from Euler-Maruyama Method for fixed parameters and try to estimate them, later mimic the process on the actual transformed series y_t .

The Euler-Maruyama Method gives the discretization:

$$\Delta x_t = \theta(\mu - x_t)\Delta t + \sigma \Delta B_t, \quad (23)$$

where $\Delta x_t, \Delta B_t$ are changes in x_t, B_t over Δt respectively. For optimization purposes we will be using the Nelder-Mead Algorithm, the reason for choosing this specific algorithm is that it doesn't compute any gradients or derivative of the objective function hence avoiding any computational approximations.

To see if the implementation works, we simulated a sample path of mean reversion process with parameters $\theta = 0.3, \mu = 3, \sigma = 1$, for $T = 1000$ and $\Delta t = 0.1$. For these values of the parameters the Log-likelihood is 88428.6454. For the optimization purposes, we assumed that all the parameters fall in the interval $(0.1, 9)$ and we proposed the initial solutions as $(\theta, \mu, \sigma) = (6, 8, 4)$. The obtained estimates using MLE method were $\hat{\theta}_{MLE} = 0.2984, \hat{\mu}_{MLE} = 3.032, \hat{\sigma}_{MLE} = 1.001$, which are pretty close to actual values. So, we can conclude that the distribution, likelihood function both are correct and the Nelder-Mead Algorithm works for optimizing the likelihood function. Now we start with estimating the parameters for the actual transformed series y_t .

3.3 Results

Due to large size of data, more than 11 times that of simulation size, we normalize the series x_t between $[1, 2]$ using a minmax scaler, to avoid numerical overflow. For the optimization function we decided to consider the following ranges for the parameters: $\theta \in [10^{-32}, 100], \mu \in [0.5, 2.5], \sigma \in [10^{-32}, 100]$ and we started by proposing initial solutions as $(\theta, \mu, \sigma) = (25, 2.3, 0.1)$ which were arbitrarily selected, the obtained optimal solution was $(\theta', \mu', \sigma') = (10^{-32}, 1.455, 0.1316)$, the solution was quite strange, since the θ estimate had hit the lower bound, implying that the algorithm got stuck.

But we had verified the algorithm on simulated data. This led us to proceed using a grid-search structure for running the optimization. Instead of fixing our initial solutions, we run the optimization iteratively on various initial solutions, compare the obtained maximum log-likelihood values and accept the solution with maximum log-likelihood. Let $(\theta, \mu, \sigma) = (m, n, p)$, where $m \in \{1, 2, \dots, 100\}, n \in \{1, 1.1, \dots, 2.0\}, p \in \{1, 2, \dots, 100\}$, for each combination of (m, n, p) we run the Nelder-Mead Algorithm with (m, n, p) as initial proposed solution, record the obtained optimal solution along with the value of log-likelihood at the obtained optimal solution. Once we have recorded the values of log-likelihoods, we compare them and find the largest log-likelihood and declare the corresponding optimal solutions as Maximum Likelihood Estimates.

After implementing this, we found that the maximum log-likelihood value was at $\theta = 0.9975, \mu = 1.493, \sigma = 0.1654$ when started with initial solutions as $m = 6, n = 1, p = 51$, with the maximum log-likelihood value as 9203.6840. So, we get the Maximum Likelihood Estimates for estimate y_t under mean reversion process as $\hat{\theta}_{MLE} = 0.9975, \hat{\mu}_{MLE} = 1.493, \hat{\sigma}_{MLE} = 0.1654$.

Now that we have the estimates, we will be proceeding to evaluating the model. For evaluation purposes we will use the funding rates data on 25th December 2024 and calling it $y_t^{(f)}$. After we have appropriately transformed the series, we will judge the model using 2 methods, first Monte Carlo Simulations, second One-step Prediction.

We will generate 10 sample paths of OU process with given MLE parameters, and consider their mean to judge the goodness of fit of the model. We will be looking at statistics like Mean Squared Error, Root Mean Squared Error, Mean Absolute Error, Mean Absolute Percentage Error and plots for residual analysis like, a plot of residuals over time, histogram of residuals and QQ plot for checking normality of residuals.

3.3.1 Monte Carlo Simulations

We start with 10 sample paths of size 1440 which is equal to number of minutes in a day. Figure 9 shows the plot of generated sample paths along with the actual values of series $y_t^{(f)}$. Once we generate the sample paths, we take the cross-sectional mean to get a vector of size (1440,1). Figure 10 shows the plot of $y_t^{(f)}$ along with Monte Carlo mean vector.

The residual analysis in Figure 11, Figure 12, Figure 13 show that the residuals contribute largely in explanation of the variance in the series $y_t^{(f)}$, also we see that the residuals are similar to a normal distribution but deviate a lot near the tails. The goodness of fit metrics also tell the same story, $MSE = 0.0411$, $RMSE = 0.2028$, $MAE = 0.1675$, $MAPE = 0.1214$, i.e. most variance in the series is explained by the residuals, hence the Mean vector of Monte-Carlo simulations is not a good predictor.

3.3.2 One-Step Prediction

As for one step prediction, we will try to predict the next value based on current value. Note that Equation 22 implies:

$$x_{t_{n+1}} = \mu + (x_{t_n} - \mu)e^{-\theta\Delta t} + \sigma\sqrt{\frac{1 - e^{-2\theta\Delta t}}{2\theta}}Z, \quad Z \sim N(0, 1). \quad (24)$$

$$\implies \hat{x}_{t_{n+1}} = \mu + (x_{t_n} - \mu)e^{-\theta\Delta t}. \quad (25)$$

The distribution also allows us to calculate a confidence interval for our estimate as (after setting $\Delta t = 1$):

$$\left(\hat{x}_{t+1} - 1.96\sigma\sqrt{\frac{1 - e^{-2\theta}}{2\theta}}, \hat{x}_{t+1} + 1.96\sigma\sqrt{\frac{1 - e^{-2\theta}}{2\theta}}\right), \quad (26)$$

for a 95% level of significance. Figure 14 shows the One-step predictors, the 95% CI of the predictors along with the true values of $y_t^{(f)}$. To take a closer look, we also plotted the same thing only for the first 200 values in Figure 15. Figure 16, Figure 17, Figure 18 shows the residual analysis of the residuals of one-step predictors, we can see that residuals are quite large ranging in $(-0.6, 0.5)$ hence explaining majority of the variance in $y_t^{(f)}$, and again there's a deviation from normality but it is lesser than that of Monte Carlo. Further we can see an improvement in the goodness of fit metrics as $MSE = 0.0326$, $RMSE = 0.1807$, $MAE = 0.1472$, $MAPE = 0.1057$. Although these are improvements, they are not good enough.

4 OU Process with Gaussian Compounded Jumps

4.1 Theory

The SDE of OU Process with Gaussian Compounded Jumps is given by:

$$dX_t = \theta(\mu - X_t)dt + \sigma dB_t + dJ_t, \quad (27)$$

where θ is the speed of mean reversion and J_t is a jump process in which the jump size is governed by $Y_i \sim N(\mu_j, \sigma_j^2)$ and the number of jumps is governed independently by N_t a Poisson Process with intensity λ i.e. $dJ_t = Y_i dN_t$.

The solution and the discretization of OUJ process is directly a result of the discretization of the Mean Reversion process:

$$\Delta X_{t_{n+1}} = \theta(\mu - X_{t_n})\Delta t_{n+1} + \sigma \Delta B_{t_{n+1}} + \Delta J_{t_{n+1}}. \quad (28)$$

The transition probabilities depend on $\Delta N_{t_{n+1}} = k$, note that since we are working with a discretised version, no doubt $dN_t \in \{0, 1\}$, but $\Delta N_{t_{n+1}} \in \mathbb{N}_0$. This implies $\Delta J_{t_{n+1}} = \sum_{i=0}^k Y_i$, $Y_i \sim N(\mu_j, \sigma_j^2)$, i.e. $\Delta J_{t_{n+1}} \sim N(k\mu_j, k\sigma_j^2) \implies$

$$f_k(X_{t_{n+1}}|X_{t_n}, N_{t_n} = k) = \frac{1}{\sqrt{2\pi(\sigma^2\Delta t_{n+1} + k\sigma_j^2)}} \exp\left(-\frac{(\Delta X_{t_{n+1}} - \theta(\mu - X_{t_n})\Delta t_{n+1} - k\mu_j)^2}{2(\sigma^2\Delta t_{n+1} + k\sigma_j^2)}\right). \quad (29)$$

So, the total transition probability is:

$$f(X_{t_{n+1}}|X_{t_n}) = \sum_{k=0}^{\infty} p(k; \lambda\Delta t_{n+1}) \cdot f_k. \quad (30)$$

where $p(k; \lambda\Delta t)$ is the value of pmf of Poisson with intensity $\lambda\Delta t$ at k . We now discuss the MCMC Metropolis Hastings algorithm in context of OU with Gaussian Compounded Jumps.

4.2 Estimation of Parameters using Metropolis Hastings

Now we will proceed with the Metropolis Hastings Algorithm to estimate the parameters. Essentially the algorithm tries to sample from the posterior using another density and at each step checks if the sample from the "proposal density" is in accordance with the posterior. Mathematically following is the algorithm:

Let's say we are interested in drawing samples from $\pi(x)$,

Step 1: Proposal Step Draw a candidate state x^* from a proposal distribution:

$$x^* \sim q(x^* | x_{t-1}). \quad (31)$$

Step 2: Compute Acceptance Probability The acceptance probability is given by:

$$\alpha = \min\left(\frac{\pi(x^*)q(x_{t-1} | x^*)}{\pi(x_{t-1})q(x^* | x_{t-1})}, 1\right). \quad (32)$$

Step 3: Accept or Reject Draw a uniform random variable:

$$u_t \sim U(0, 1). \quad (33)$$

If $u_t \leq \alpha$, accept x^* and set:

$$x_t = x^*. \quad (34)$$

Otherwise, reject x^* and keep:

$$x_t = x_{t-1}. \quad (35)$$

Step 4: Iterate Repeat the process for a sufficiently large number of steps to generate a Markov chain that approximates the target distribution $\pi(x)$.

Once we have obtained a sample, we burn a certain amount of initial values which were away from the limit of the sequence before proposing the mean of sample as an estimate. Here we have choosen the burn-in period as 0.4. First we will simulate an OU Process with Gaussian compounded Jumps and then use the MH algorithm to estimate the parameters. Once we have verified our method of estimation we move forward with series y_t .

4.3 Simulation Study

We will simulate the OUJ process using the discretised form of the SDE as in Equation 28, note that if N_t is a Poisson process then $\Delta N_{t_{n+1}} = N_{t_{n+1}} - N_{t_n} \sim \text{Poisson}(\lambda \Delta t_{n+1})$, we use this result to simulate the Poisson process and hence the jumps. We have chosen $T = 1000$, $\Delta t = 0.01$, $X_0 = 0$, $\theta = 1.5$, $\mu = 0$, $\sigma = 0.3$, $\lambda = 2$, $\mu_j = 0$, $\sigma_j = 3$ to simulate the OUJ process. For computational purposes we have put an upper limit on $\Delta N_{t_n} = k \leq 6$ since $\lambda \Delta t = 0.02$ and $\mathbb{P}(\Delta N_{t_n} > 6) \approx 10^{-15}$.

Table 3 gives the the further details on the Metropolis Hastings algorithm like choice of priors, proposals, and obtained estimates. Figure 19, Figure 21, Figure 23, Figure 25, Figure 27, and Figure 29 show the MH Chains of respective parameters, we can clearly see the convergence in each of them. Moreover Figure 20, Figure 22, Figure 24, Figure 26, Figure 28, and Figure 30 are the histograms of chains after the burn-in period of 0.3. The clear convergence in each parameter shows that the MH algorithm works with above given priors, proposals, and computational considerations.

4.4 Estimation on y_t

To use Metropolis Hastings on the actual series y_t we need to make an adjustment. When we were working with simulated data, one can set Δt to their choosing, but when we work with observed data, the granularity is fixed. So we change our unit time from one minute to twenty minutes, this will let us to set $\Delta t_n = 0.05$. This is necessary because the Euler-Maruyama Method holds only to small Δt , and since our likelihood and posterior are based on Euler Maruyama. We will also rescale the series y_t using a minmax scaler so that $y_t \in [1, 2]$.

Table 4 gives the the further details on the Metropolis Hastings algorithm like choice of priors, proposals, and obtained estimates. Figure 31, Figure 33, Figure 35, Figure 37, Figure 39, and Figure 41 show the MH Chains of respective parameters. Moreover Figure 32, Figure 34, Figure 36, Figure 38, Figure 40, and Figure 42 are the histograms of chains after the burn-in period.

The plots of chains and their histograms suggests that the chain isn't evolving quickly, that is the acceptance of new samples is quite rare. The reason for this malfunction in the MH algorithm is poor variance control of proposals. Since, the proposal of a certain parameters is forced to have its expectation as the last state of that parameter, and if the proposal distribution is an exponential

distribution, we can't adjust its variance. The reason to adjust variance lies in the mechanism of MH algorithm. In every iteration, we propose a new state for the chain for a distribution whose mean is the last state, if the variance is too high then the proposed new state is far away from the last state, which results in poor exploration of parameter space in a granular sense, whereas if the variance is too low, the proposed new state isn't much different than the last state and hence the chain could take a lot of iterations to reach the optimal state.

Based on the above discussion we make two changes, first we change all exponential proposals to Gamma proposal distributions, to get a better control on the variance, second we increase the chain size from 5000 to 15000. Table 5 gives the further details on the Metropolis Hastings algorithm like choice of priors, proposals, and obtained estimates. Figure 43, Figure 45, Figure 47, Figure 49, Figure 51, and Figure 53 show the MH Chains of respective parameters. Moreover Figure 44, Figure 46, Figure 48, Figure 50, Figure 52, and Figure 54 are the histograms of chains after the burn-in period of 0.3. The plots of the chains of all parameters and the histograms of the chain show a clear convergence of the chains for each parameter, with following estimated parameter values:

$$(\hat{\theta}^{(MH)}, \hat{\mu}^{(MH)}, \hat{\sigma}^{(MH)}, \hat{\lambda}^{(MH)}, \hat{\mu}_j^{(MH)}, \hat{\sigma}_j^{(MH)}) = (10.883, 1.493, 0.183, 7.756, 0.010, 0.056). \quad (36)$$

Unfortunately the generation of the Metropolis Hastings chains took approximately 25 hours of computation. We could have made this work in capital markets since they are open for 6-8 hours in a day, and we can reduce length of y_t to get the runtime down to 12-14 hours. But the cryptocurrency market is open for 24 hours, due to this reason we need to look for methods which are computationally less expensive.

5 Jump Analysis

We will now take a new approach to estimating parameters and drawing out inferences on the funding rates. First we will separate the jumps from an OUJ process and estimate the parameters of OU part and the jump part separately. Let $(\hat{\theta}^{(0)}, \hat{\mu}^{(0)}, \hat{\sigma}^{(0)}, \hat{\lambda}^{(0)}, \hat{\mu}_j^{(0)}, \hat{\sigma}_j^{(0)})$ be the estimates obtained by separately estimating the parameters of two processes, note that these estimates won't be same as the MH estimates, since separating the process will result different variances and mean reversion speeds due to the drift term being dependent on the process that we are modeling.

5.1 Lee Mykland Test

To separate the process, firstly we discuss the Lee-Mykland test for jumps which is a non-parametric test. The Lee-Mykland test defines the statistic:

$$\mathcal{L}(i) = \frac{\log(S(t_i)/S(t_{i-1}))}{\hat{\sigma}_K(i)}, \quad (37)$$

$$\hat{\sigma}_K^2(i) = \frac{1}{K-2} \sum_{j=i-K+2}^{i-1} |\log(S(t_i)/S(t_{i-1}))| \cdot |\log(S(t_{i-1})/S(t_{i-2}))|, \quad (38)$$

for the underlying process $d\log(S_t) = \theta(\mu - S_t)dt + \sigma dB_t + dJ_t$, but we are not modeling $\log(f_t)$, hence for $df_t = \theta(\mu - f_t)dt + \sigma dB_t + dJ_t$ the test transforms to:

$$\mathcal{L}(i) = \frac{f_{t_i} - f_{t_{i-1}}}{\hat{\sigma}_K(i)}, \quad (39)$$

$$\hat{\sigma}_K^2(i) = \frac{1}{K-2} \sum_{j=i-K+2}^{i-1} |f_{t_i} - f_{t_{i-1}}| \cdot |f_{t_{i-1}} - f_{t_{i-2}}|. \quad (40)$$

The Lee-Mykland test proposes that under null hypothesis of no-jump, the test statistic has an asymptotically normal distribution. The reason for choosing Lee Mykland test over other tests is its ability to give inference for each and every point rather than a global inference. We have simulated an OUJ process with parameters: $(\theta, \mu, \sigma, \lambda, \mu_j, \sigma_j) = (1.0, 3.0, 0.3, 1.0, 0.0, 2.0)$, $T = 1000$, $\Delta t = 0.01$, $X_0 = 0.0$. We ran a Lee Mykland test at 90%, 95%, 99% level of significance and the results are in [Table 6](#). We can see that the test has a high false positive rate, which would lead to bad separation and hence bad estimates.

5.2 A test to detect Jumps in Stochastic Models

We now propose a test for detecting jumps in stochastic models, then compare the results with the Lee-Mykland test and proceed to separate the jump process from f_t . The test uses the Euler-Maruyama method for discretization, hence it is important to keep Δt very small. Further, we also have a constraint on the maximum number of jumps in an interval of size Δt which is to be set by the user, let us call this maximum number of jumps k_{max} . Using the Euler-Maruyama discretization in [Equation 23](#) and [Equation 28](#) we define the following notation:

$$\Delta X_{t_{n+1}} \sim OU^{(n)}(\theta, \mu, \sigma) = N(\theta(\mu - X_{t_n})\Delta t, \sigma^2 \Delta t) = N(\mu_{0,n}, \sigma_{0,n}^2), \quad (41)$$

$$\Delta X_{t_{n+1}} \sim OUJ_{k_{max}}^{(n)}(\theta, \mu, \sigma, \lambda, \mu_j, \sigma_j) \quad (42)$$

$$= N\left(\frac{\sum_{k=0}^{k_{max}} p(k, \lambda \Delta t) [\mu_{0,n} + k\mu_j]}{\sum_{k=0}^{k_{max}} p(k, \lambda \Delta t)}, \frac{\sum_{k=0}^{k_{max}} p(k, \lambda \Delta t)^2 [\sigma_{0,n}^2 + k\sigma_j^2]}{\sum_{k=0}^{k_{max}} p(k, \lambda \Delta t)^2}\right), \quad (43)$$

$$= N(\mu_{1,n}, \sigma_{1,n}^2), \quad (44)$$

where $p(k, \lambda_0)$ is the Poisson pmf with mean λ_0 evaluated at k and $0 \leq \varepsilon \leq 1$. The test we propose is for the following set of hypothesis:

$$H_0 : \Delta X_{t_{n+1}} \sim OU^{(n)}(\theta, \mu, \sigma), \quad (45)$$

$$H_1^\varepsilon : \Delta X_{t_{n+1}} \sim \varepsilon \cdot OU^{(n)}(\theta, \mu, \sigma) + (1 - \varepsilon) \cdot OU_{k_{max}}^{(n)}(\theta, \mu, \sigma, \lambda, \mu_j, \sigma_j), \quad (46)$$

and the test statistic is:

$$T_{n+1} = \frac{\Delta X_{t_{n+1}} - \mu_{0,n}}{\sigma_{0,n}} \sim N(0, 1) \text{ under } H_0, \quad (47)$$

with a two-sided rejection region.

Proposition: Under the above setup, the power of the test is a decreasing function of ε .

Proof:

Let $Y = \Delta X_{t_{n+1}}$,

$$\begin{aligned} \text{Power} &= P(T > z_{1-\alpha/2} | H_1^\varepsilon) + P(T < -z_{1-\alpha/2} | H_1^\varepsilon) \\ &= \varepsilon \cdot [P(T > z_{1-\alpha/2} | T \sim N(0, 1)) + P(T < -z_{1-\alpha/2} | T \sim N(0, 1))] \\ &\quad + (1 - \varepsilon) \cdot [P(T > z_{1-\alpha/2} | T \sim N(\delta, \tau^2)) + P(T < -z_{1-\alpha/2} | T \sim N(\delta, \tau^2))], \end{aligned}$$

for $\delta = \frac{\mu_{1,n} - \mu_{0,n}}{\sigma_{0,n}}$, $\tau = \frac{\sigma_{1,n}}{\sigma_{0,n}}$ since,

$$\begin{aligned} T &= \frac{Y - \mu_{0,n}}{\sigma_{0,n}} \\ &= \frac{\sigma_{1,n}}{\sigma_{0,n}} \left(\frac{X - \mu_{1,n}}{\sigma_{1,n}} \right) + \frac{\mu_{1,n} - \mu_{0,n}}{\sigma_{0,n}} \\ &\sim N\left(\frac{\mu_{1,n} - \mu_{0,n}}{\sigma_{0,n}}, \frac{\sigma_{1,n}^2}{\sigma_{0,n}^2}\right) \text{ under } H_1^{\varepsilon=0}. \end{aligned}$$

This implies

$$\text{Power} = \varepsilon \cdot \alpha + (1 - \varepsilon) \cdot \left(1 - \Phi\left(\frac{z_{1-\alpha/2} - \delta}{\tau}\right) + \Phi\left(\frac{-z_{1-\alpha/2} - \delta}{\tau}\right) \right). \quad (48)$$

Note that

$$\begin{aligned} \mu_{1,n} &= \mu_{0,n} + \mu_j \frac{\sum_{k=0}^{k_{max}} k \cdot p(k, \lambda \Delta t)}{\sum_{k=0}^{k_{max}} p(k, \lambda \Delta t)}, \quad \sigma_{1,n}^2 = \sigma_{0,n}^2 + \sigma_j^2 \frac{\sum_{k=0}^{k_{max}} k^2 \cdot p(k, \lambda \Delta t)^2}{\sum_{k=0}^{k_{max}} p(k, \lambda \Delta t)^2}, \\ \implies \delta &= \frac{\mu_{1,n} - \mu_{0,n}}{\sigma_{0,n}} \geq 0, \quad \tau = \frac{\sigma_{1,n}}{\sigma_{0,n}} \geq 1. \end{aligned}$$

We rewrite,

$$\begin{aligned}
D(\delta, \tau) &= 1 - \Phi\left(\frac{z_{1-\alpha/2} - \delta}{\tau}\right) + \Phi\left(\frac{z_{\alpha/2} - \delta}{\tau}\right) \\
&= 1 - \Phi(u_1(\delta, \tau)) + \Phi(u_2(\delta, \tau)), \\
\Rightarrow \frac{\partial \Phi(u_1)}{\partial \delta} &= -\frac{1}{\tau} \phi\left(\frac{z_{1-\alpha/2} - \delta}{\tau}\right), \quad \frac{\partial \Phi(u_2)}{\partial \delta} = -\frac{1}{\tau} \phi\left(\frac{-z_{1-\alpha/2} - \delta}{\tau}\right) \\
\Rightarrow \frac{\partial D(\delta, \tau)}{\partial \tau} &= -\left(-\frac{1}{\tau} \phi\left(\frac{z_{1-\alpha/2} - \delta}{\tau}\right)\right) + \left(-\frac{1}{\tau} \phi\left(\frac{-z_{1-\alpha/2} - \delta}{\tau}\right)\right), \\
&= \frac{1}{\tau} \phi\left(\frac{z_{1-\alpha/2} - \delta}{\tau}\right) - \frac{1}{\tau} \phi\left(\frac{-z_{1-\alpha/2} - \delta}{\tau}\right) \\
&= \frac{1}{\tau} \phi\left(\frac{z_{1-\alpha/2} - \delta}{\tau}\right) - \frac{1}{\tau} \phi\left(\frac{z_{1-\alpha/2} + \delta}{\tau}\right) \\
&< 0 \quad \because \delta \geq 0, \phi(x) \text{ is decreasing for } x \geq 0, \\
\Rightarrow D(\delta, \tau) &> D(0, \tau) = 2\left(1 - \phi\left(\frac{z_{1-\alpha/2}}{\tau}\right)\right) \\
&> 2(1 - \phi(z_{1\alpha/2})) \quad (\tau \geq 1) \\
&> \alpha \quad \forall \delta \geq 0.
\end{aligned}$$

As $D(\delta, \tau) > \alpha \forall \delta > 0$, implies the derivative of Power with respect to ε is negative, hence Power decreases with increase in ε . □

The claim is intuitive since with decrease in ε the impurity of OU in H_1^ε also decreases making the processes under null and alternative more distinctive. Note that, we have used the distributional properties of OU, OUI only to prove that $\delta \geq 0, \tau \geq 1$, hence we can generalize the result in following manner:

Proposition: Consider an Ito process $dX_{t_{n+1}} = \mu(X_{t_n}, t_n)dt + \sigma(X_{t_n}, t_n)dB_{t_n}$, and its discretized version $\Delta X_{t_{n+1}} \sim N(\mu(X_{t_n}, t_n), \sigma^2(X_{t_n}, t_n)\Delta t_n)$. To test the hypotheses

$$\begin{aligned}
H_0 : \Delta X_{t_{n+1}} &\sim N(\mu(X_{t_n}, t_n), \sigma^2(X_{t_n}, t_n)\Delta t_n) = N(\mu_{0,n}, \sigma_{0,n}^2), \\
H_1^\varepsilon : \Delta X_{t_{n+1}} &\sim N\left(\frac{\sum_{k=0}^{k_{max}} p(k, \lambda\Delta t)[\mu_{0,n} + k\mu_j]}{\sum_{k=0}^{k_{max}} p(k, \lambda\Delta t)}, \frac{\sum_{k=0}^{k_{max}} p(k, \lambda\Delta t)^2[\sigma_{0,n}^2 + k\sigma_j^2]}{\sum_{k=0}^{k_{max}} p(k, \lambda\Delta t)^2}\right), \\
&= N(\mu_{1,n}, \sigma_{1,n}^2),
\end{aligned}$$

we use the test statistic as:

$$T_{n+1} = \frac{\Delta X_{t_{n+1}} - \mu_{0,n}}{\sigma_{0,n}} \sim N(0, 1) \text{ under } H_0,$$

then the power of the test is a decreasing function of ε if

$$\delta = \frac{\mu_{1,n} - \mu_{0,n}}{\sigma_{0,n}} \geq 0, \quad \tau = \frac{\sigma_{1,n}}{\sigma_{0,n}} \geq 1.$$

To compare it with Lee-Mykland test, we use the above method for $H_0 : \Delta X_{t_{n+1}} \sim OU^{(n)}(\theta, \mu, \sigma)$, $H_1^{\varepsilon=0} : \Delta X_{t_{n+1}} \sim OUJ_{k_{max}}^{(n)}(\theta, \mu, \sigma, \lambda, \mu_j, \sigma_j)$. We also use the same simulated an OIJ process with parameters: $(\theta, \mu, \sigma, \lambda, \mu_j, \sigma_j) = (1.0, 3.0, 0.3, 1.0, 0.0, 2.0)$, $T = 1000$, $\Delta t = 0.01$, $X_0 = 0.0$, and the test at 90%, 95%, 99% level of significance. [Table 7](#) summarizes the results for the above proposed test. In comparison with [Table 6](#), the above proposed test gives far better results than the Lee-Mykland Test.

Now we use the test at 99% level on the funding rates series f_t . Initially we consider a thresholding cutoff of 3.5 times the standard deviation of ΔX to get a empirical estimates of θ, μ, σ using AR(1) process. For the test we need to calculate $\mu_{0,n}, \sigma_{0,n}$ which depend only on the parameters θ, μ, σ and $X_{t_n}, \Delta t$. Using this test we were able to identify 1449, in the 8-day period 17-12-2024 to 24-12-2024. [Figure 55](#) shows the identified jumps. After identifying the jumps, we separate the two processes $dX_t = \theta(\mu - X_t)dt + \sigma dB_t$, $dJ_t = YdNt$, $df_t = dX_t + dJ_t$. Then we find estimates of θ, μ, σ using AR(1) estimates and the estimates of λ, μ_j, σ_j by taking the ratio of number of obtained jumps to that of the total series length, mean of the obtained jumps, and standard deviation of the obtained jumps respectively. The estimates are:

$$(\hat{\theta}, \hat{\mu}, \hat{\sigma}, \hat{\lambda}, \hat{\mu}_j, \hat{\sigma}_j) = (14.242, 1.519, 0.120, 4.586, 0.0, 0.064) \quad (49)$$

For forecasting purposes, we propose a method which is a mix of one-step prediction as in [Section 3.3.2 One-Step Prediction](#) for the OU part of forecast and perform a Monte Carlo simulation for the jump part, we have considered a simulation size of 1000. We estimated the parameters on data from 17-12-2024 to 24-12-2024 and now test the fit on 25-12-2024. [Figure 56](#) shows the simulated paths, while [Figure 57](#) shows the prediction on f_t during 25-12-2024, [Figure 58](#) gives a closer look into the predictions. [Table 8](#) compares various fit metrics of the OIJ model with that of OU model with 1-step prediction, which clearly shows that the OIJ model is a better fit than the OU model.

6 Conclusion and Further Explorations

We started with some basic descriptive study in which we looked into stationarity, autocorrelation properties, and distributional properties wherein we proposed a transformation to convert the distribution of f_t to a Log-Normal. Then we looked into OU process and the estimation of parameters of the OU process using Maximum Likelihood Estimation using Nelder-Mead algorithm which was chosen to eliminate the need of gradient computations. The Nelder-Mead algorithm had difficulty when working with the transformed series y_t , so we implemented a grid-search method upon the Nelder-Mead algorithm. Further we performed a goodness of fit analysis, which revealed that when modelling with OU process most of the variance in the series is explained by the residuals which indicate a poor fit.

The residual analysis motivated us to include jumps in our model, but due to the complex nature of the likelihood function, maximum likelihood estimation was no longer a choice. So we used MCMC Metropolis Hastings Algorithm for estimation of the parameters of the OUJ process, which was a computationally expensive method and for that reason we need to look for other methods of estimation. So we decided to use statistical tests to separate the jump process and the OU process. The most common test is the Lee-Mykland test proposed in 2008, this test had a high false positive rate or Type 1 errors. This could lead to improper separation, hence giving bad estimates. So, we proposed a new test based on the Euler-Maruyama discretization. This test had much lower type 1 errors, giving us a better separation, finally we use AR(1) process to estimate the OU parameters and empirical estimates for the jump part using data from 17-12-2024 to 24-12-2024, to further forecast the process f_t on 25-12-2024. The mean absolute percentage error for OU model was 10.57%, while the mean absolute percentage error for OUJ model was 1.06% indicating a much better fit. The parameters obtained for 20-min frequency in Equation 49 shows that the processes reverts back to its mean at a much faster rate, while $\sigma_j < \sigma$ we can see that the frequency of jumps is quite high, also the jump sizes are unbiased since the mean size of jump is 0.

Since the test is computationally efficient, one can use the test in high frequency trading algorithms to make informed decisions for processes with jumps given enough granularity of data is available. Further a dynamic approach can be implemented on a rolling basis to estimate the parameters on past 7-8 days and use the estimated parameters to forecast the funding rates for the upcoming day. This rolling method can help preserve enough recent trend. For predicting jumps one can make use of jumps in premium index, since funding rates are weighted averages of premium index hence if there is significant jump activity in premium index (which can be found using the proposed test,) recently then due to the decaying weights one can argue for a upcoming possible jump in funding rates.

Figures and Tables

Table 1: Elementary Statistics for Funding rates in 10^{-4} .

Minimum	-7.64
25%-tile	0.08
50%-tile	0.81
75%-tile	1.44
Maximum	5.04
Mean	0.66
Stan. Dev,	1.22

Table 2: Comparison of Statistics before and after Transformation

Statistics/Series	f'_t	f'_t Log-Normal	x_t	x_t Log-Normal
Skewness	1.0365	0.4539	0.1354	0.0830
Excess Kurtosis	3.3252	0.4217	0.7245	0.065
KS Density Deviation	0.0457	0	0.0178	0

Table 3: Metropolis Hastings Details for Simulation study.

Parameter	Prior	Proposal	Simulation Value	Estimated Value
θ	$\Gamma(2, 1)$	$\text{Exp}(\theta^{(t)})$	1.5	1.497
μ	$N(0, 10)$	$N(\mu^{(t)}, 0.02)$	0	0.007
σ	$\Gamma(2, 1)$	$\text{Exp}(\sigma^{(t)})$	3	2.980
λ	$\Gamma(2, 1)$	$\text{Exp}(\lambda^{(t)})$	2	2.039
μ_j	$N(0, 10)$	$N(\mu_j^{(t)}, 0.02)$	0	-0.063
σ_j	$\Gamma(2, 1)$	$\text{Exp}(\sigma_j^{(t)})$	3	3.063

Table 4: Metropolis Hastings Details for y_t .

Parameter	Prior	Proposal	Estimated Value	Standard Error
θ	$\Gamma(2, 1)$	$\text{Exp}(\theta^{(t)})$	12.222	0.353
μ	$N(0, 10)$	$N(\mu^{(t)}, 0.001)$	1.506	0
σ	$\Gamma(2, 1)$	$\text{Exp}(\sigma^{(t)})$	0.243	0.004
λ	$\Gamma(2, 1)$	$\text{Exp}(\lambda^{(t)})$	3.584	4.997
μ_j	$N(0, 10)$	$N(\mu_j^{(t)}, 0.005)$	0.188	0.002
σ_j	$\Gamma(2, 1)$	$\text{Exp}(\sigma_j^{(t)})$	0.001	0

Table 5: MH Details for y_t using updated proposals and longer chains.

Parameter	Prior	Proposal	Estimated Value	Standard Error
θ	$\Gamma(2, 1)$	$\Gamma(\theta^{(t)}, 0.02)$	10.883	0.494
μ	$N(0, 10)$	$N(\mu^{(t)}, 0.002)$	1.493	0.001
σ	$\Gamma(2, 1)$	$\Gamma(\sigma^{(t)}, 0.002)$	0.183	0.002
λ	$\Gamma(2, 1)$	$\Gamma(\lambda^{(t)}, 0.05)$	7.756	0.528
μ_j	$N(0, 10)$	$N(\mu_j^{(t)}, 0.01)$	0.010	0.002
σ_j	$\Gamma(2, 1)$	$\Gamma(\sigma_j^{(t)}, 0.004)$	0.056	0.003

Table 6: Analysis of Lee Mykland Test.

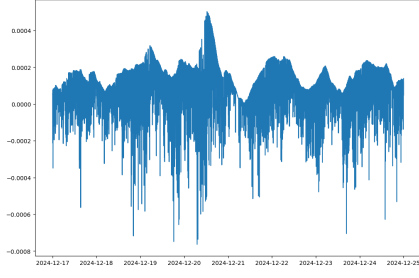
α	Total Jumps	True Positives	True Negatives	False Positives	False Negatives
90%	971	953	82136	16894	18
95%	971	949	88358	10672	22
99%	971	934	95153	3877	37

Table 7: Analysis of the proposed Test.

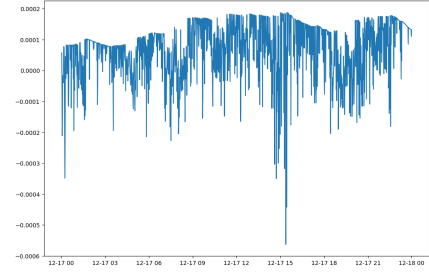
α	Total Jumps	True Positives	True Negatives	False Positives	False Negatives
90%	971	952	95273	3757	19
95%	971	947	97705	1325	24
99%	971	933	98925	105	38

Table 8: Fit metrics comparison between OU and OUJ models

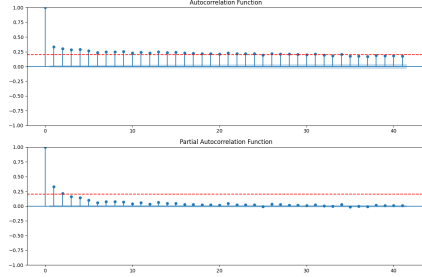
Metrics	OU (1-step)	OUJ
MSE	0.0326	0.005
RMSE	0.1807	0.0214
MAE	0.1472	0.0156
MAPE	0.1057	0.0106



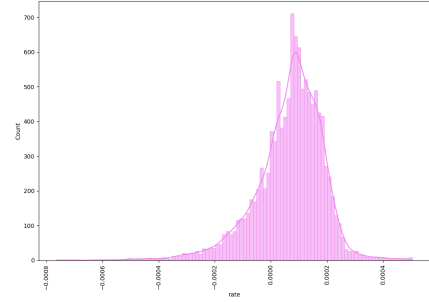
(1) Figure 1: Funding Rates plot from 17-12-2024 to 24-12-2024



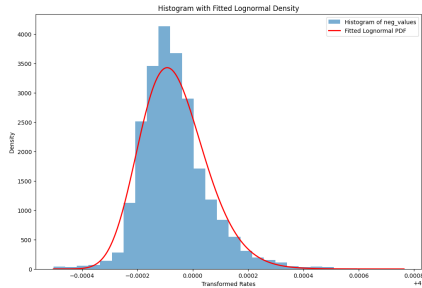
(2) Figure 2: Funding Rates plot 17-12-2024



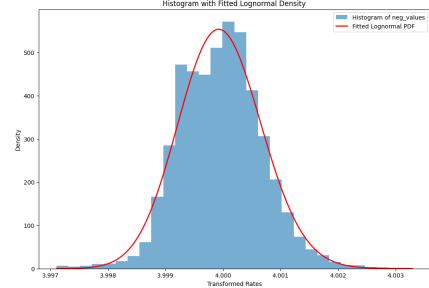
(3) Figure 3: ACF-PACF Plots



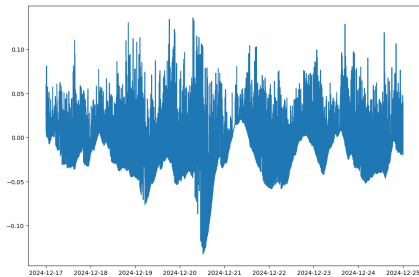
(4) Figure 4: Histogram of Funding Rates



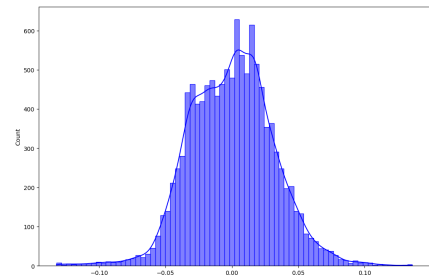
(5) Figure 5: Best fit Log-normal Density on Transformed Funding Rates f'_t



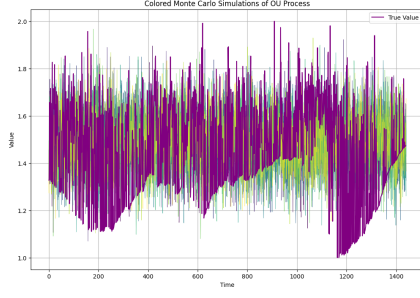
(6) Figure 6: Best fit Log-normal Density on Transformed Funding Rates x_t



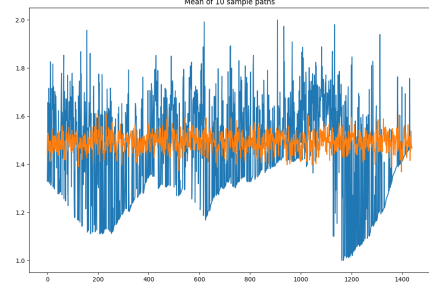
(7) Figure 7: Plot of y_t



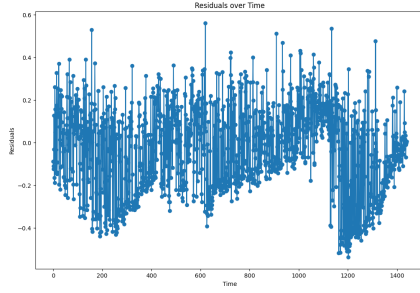
(8) Figure 8: Histogram of y_t



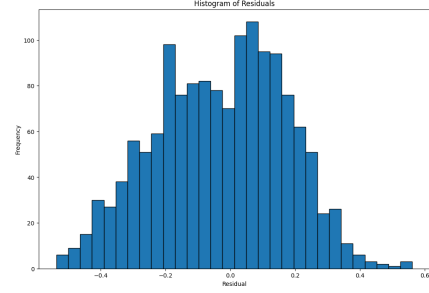
(9) Figure 9: Monte Carlo Sample paths and true $y_t^{(f)}$ values



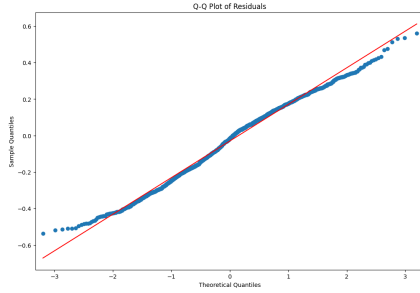
(10) Figure 10: Monte Carlo Sample paths Mean and true $y_t^{(f)}$ values



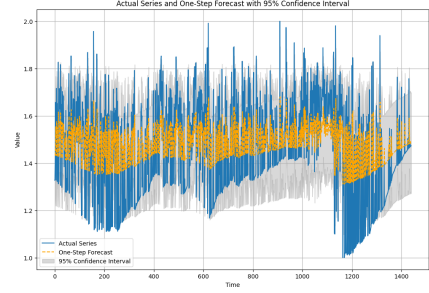
(11) Figure 11: Monte Carlo Sample paths Mean residuals vs time



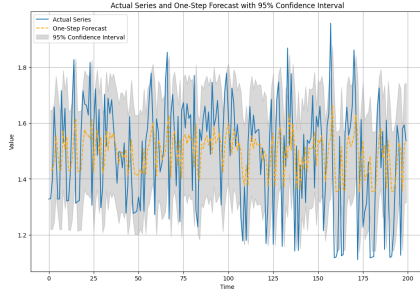
(12) Figure 12: Monte Carlo Sample paths Mean residuals Histogram



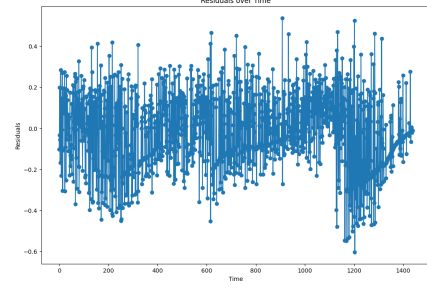
(13) Figure 13: Monte Carlo Sample paths Mean residuals QQ Plot



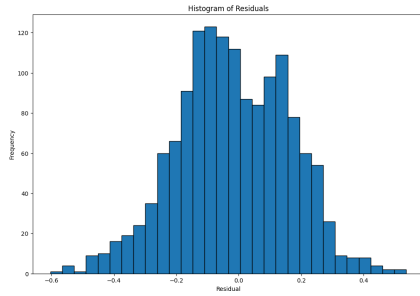
(14) Figure 14: One Step Prediction, 95% Confidence interval along with True Values



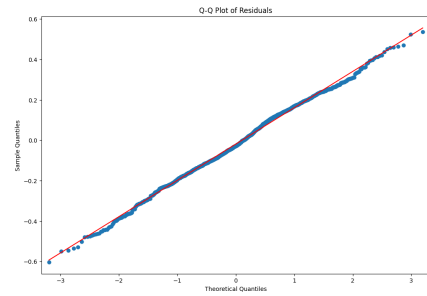
(15) Figure 15: One Step Prediction, 95% Confidence interval along with True Values for first 200 values



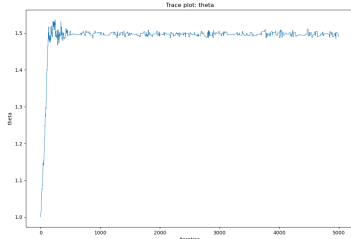
(16) Figure 16: One Step Prediction residuals vs Time



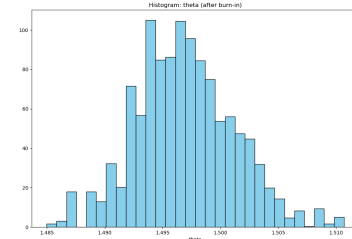
(17) Figure 17: One Step Prediction residuals Histogram



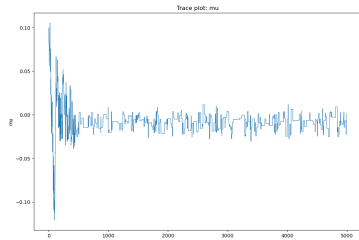
(18) Figure 18: One Step Prediction residuals QQ Plot



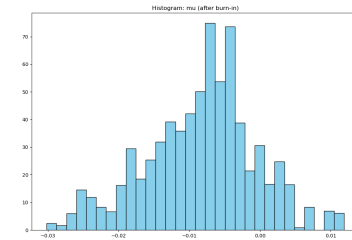
(19) Figure 19: MH Chain of θ in Simulation Study



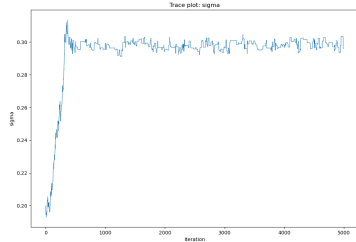
(20) Figure 20: Histogram of MH Chain of θ in Simulation Study



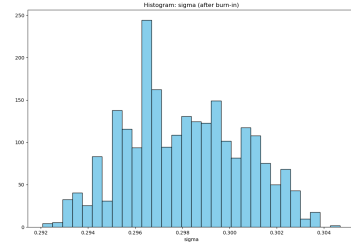
(21) Figure 21: MH Chain of μ in Simulation Study



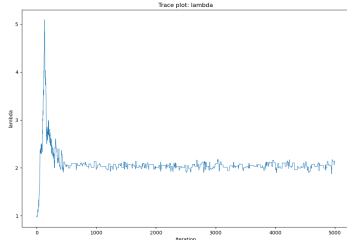
(22) Figure 22: Histogram of MH Chain of μ in Simulation Study



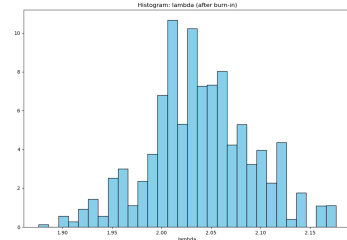
(23) Figure 23: MH Chain of σ in Simulation Study



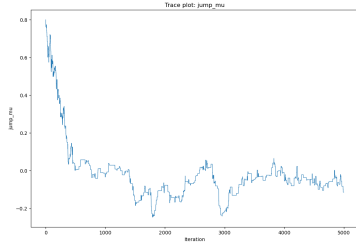
(24) Figure 24: Histogram of MH Chain of σ in Simulation Study



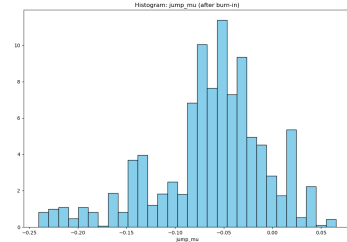
(25) Figure 25: MH Chain of λ in Simulation Study



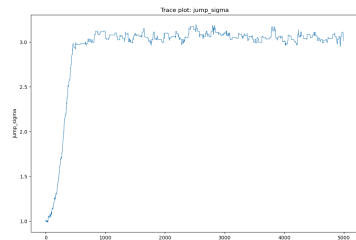
(26) Figure 26: Histogram of MH Chain of λ in Simulation Study



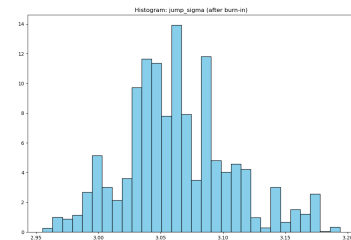
(27) Figure 27: MH Chain of μ_j in Simulation Study



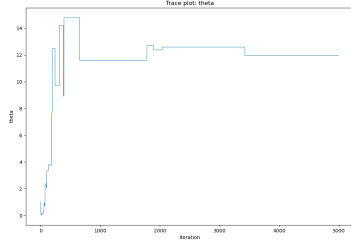
(28) Figure 28: Histogram of MH Chain of μ_j in Simulation Study



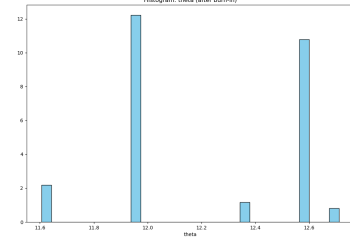
(29) Figure 29: MH Chain of σ_j in Simulation Study



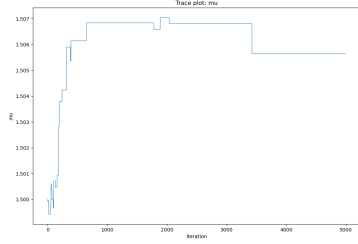
(30) Figure 30: Histogram of MH Chain of σ_j in Simulation Study



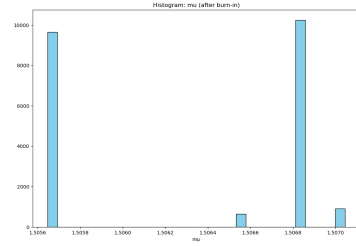
(31) Figure 31: MH Chain of θ for y_t



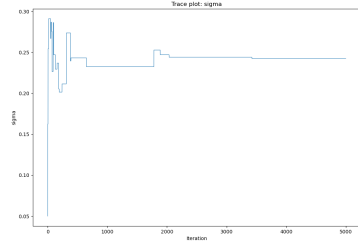
(32) Figure 32: Histogram of MH Chain of θ in for y_t



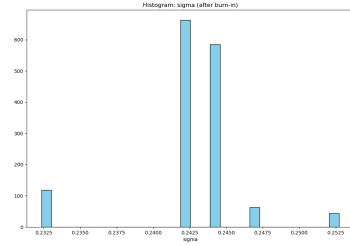
(33) Figure 33: MH Chain of μ for y_t



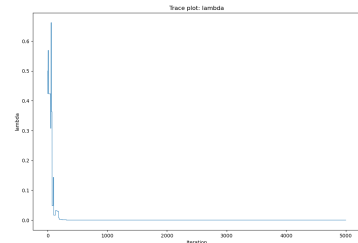
(34) Figure 34: Histogram of MH Chain of μ in for y_t



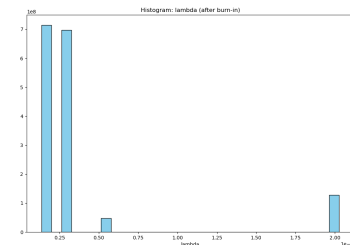
(35) Figure 35: MH Chain of σ for y_t



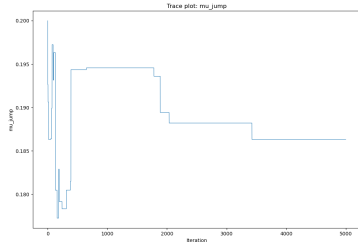
(36) Figure 36: Histogram of MH Chain of σ in for y_t



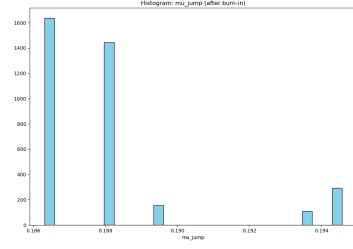
(37) Figure 37: MH Chain of λ for y_t



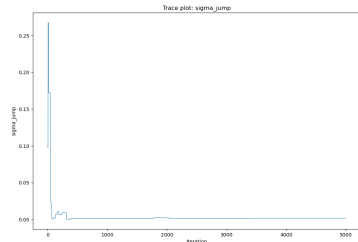
(38) Figure 38: Histogram of MH Chain of λ in for y_t



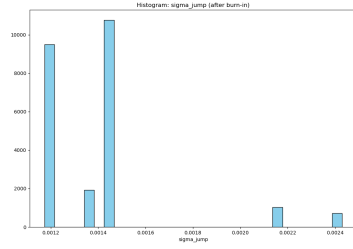
(39) Figure 39: MH Chain of μ_j for y_t



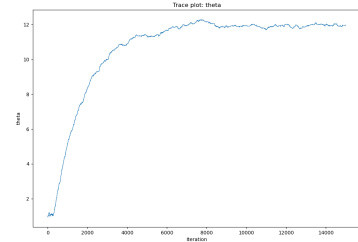
(40) Figure 40: Histogram of MH Chain of μ_j in for y_t



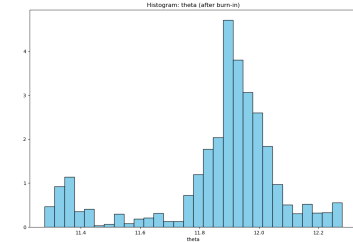
(41) Figure 41: MH Chain of σ_j for y_t



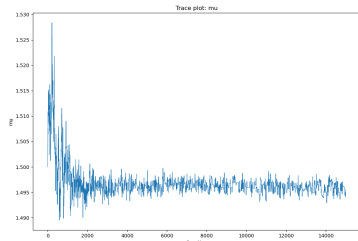
(42) Figure 42: Histogram of MH Chain of σ_j in for y_t



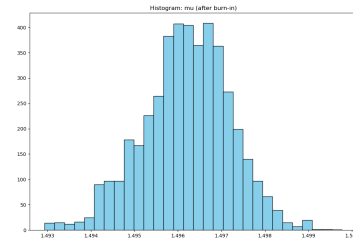
(43) Figure 43: MH Chain of θ for y_t with updated proposals



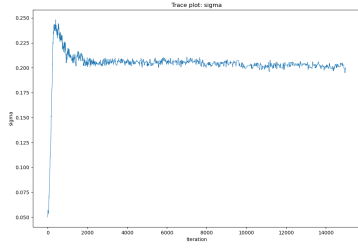
(44) Figure 44: Histogram of MH Chain of θ in for y_t with updated proposals



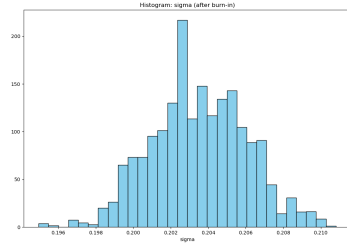
(45) Figure 45: MH Chain of μ for y_t with updated proposals



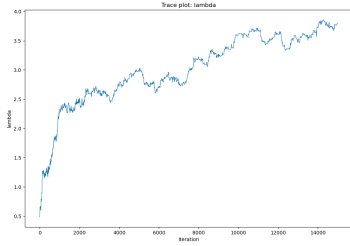
(46) Figure 46: Histogram of MH Chain of μ in for y_t with updated proposals



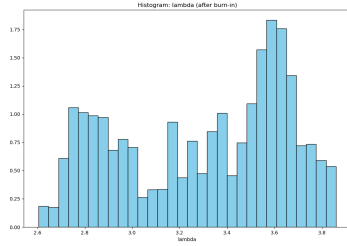
(47) Figure 47: MH Chain of σ for y_t with updated proposals



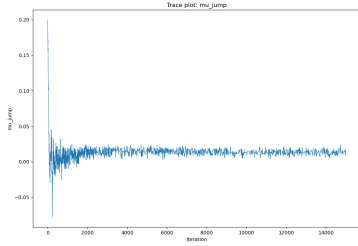
(48) Figure 48: Histogram of MH Chain of σ in for y_t with updated proposals



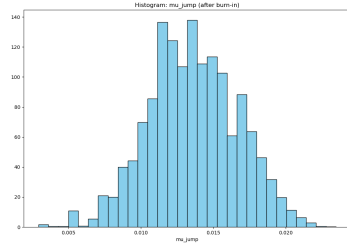
(49) Figure 49: MH Chain of λ for y_t with updated proposals



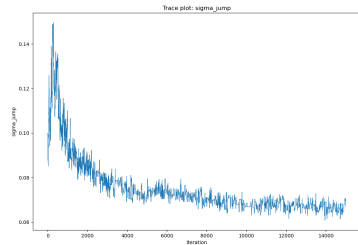
(50) Figure 50: Histogram of MH Chain of λ in for y_t with updated proposals



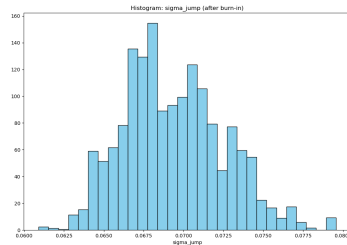
(51) Figure 51: MH Chain of μ_j for y_t with updated proposals



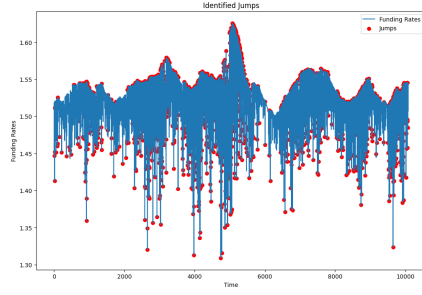
(52) Figure 52: Histogram of MH Chain of μ_j in for y_t with updated proposals



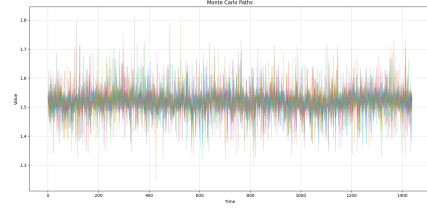
(53) Figure 53: MH Chain of σ_j for y_t with updated proposals



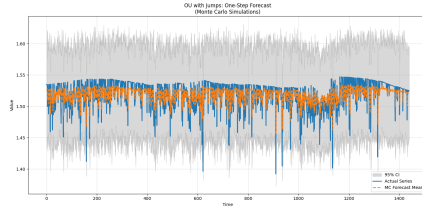
(54) Figure 54: Histogram of MH Chain of σ_j in for y_t with updated proposals



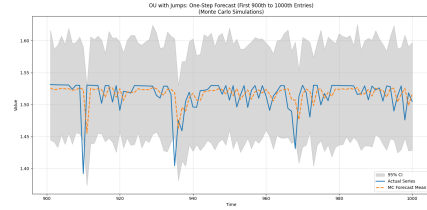
(55) Figure 55: Jumps identified by proposed test in f_t



(56) Figure 56: Monte Carlo paths in forecasting of f_t



(57) Figure 57: Forecast for f_t with confidence intervals on 25-12-2024



(58) Figure 58: Closer look at predictions of f_t on 25-12-2024 between 900-1000 observations

References

- [1] Wiersema, U. F. (2008). *Brownian Motion Calculus*. John Wiley & Sons.
- [2] Hirta, A. (2013). *Computational Methods in Finance*. CRC Press, Taylor & Francis Group.
- [3] Tankov, P. *Financial Modeling with Lévy Processes*. Centre de Mathématiques Appliquées, École Polytechnique. Lecture Notes.
- [4] Shumway, R. H., & Stoffer, D. S. (2011). *Time Series Analysis and Its Applications: With R Examples* (3rd ed.). Springer.
- [5] Tankov, P., & Voltchkova, E. *Jump-Diffusion Models: A Practitioner's Guide*. Université Paris 7 and Université Toulouse 1.
- [6] Tang, F. (2018). *Merton Jump-Diffusion: Modeling of Stock Price Data* (Bachelor's thesis). Linnaeus University.
- [7] Lee, S. S., & Mykland, P. A. (2008). Jumps in Financial Markets: A New Nonparametric Test and Jump Dynamics. *Review of Financial Studies*, 21(6), 2535–2563. <https://doi.org/10.1093/rfs/hhm056>
- [8] Dickey, D. A., & Fuller, W. A. (1979). Distribution of the Estimators for Autoregressive Time Series with a Unit Root. *Journal of the American Statistical Association*, 74(366), 427–431. <https://doi.org/10.2307/2286348>

- [9] Kolmogorov, A. N. (1933). Sulla determinazione empirica di una legge di distribuzione. *Giornale dell'Istituto Italiano degli Attuari*, 4, 83–91.
- [10] Smirnov, N. V. (1948). Table for Estimating the Goodness of Fit of Empirical Distributions. *The Annals of Mathematical Statistics*, 19(2), 279–281. <https://doi.org/10.1214/aoms/1177730256>
- [11] Maruyama, G. (1955). Continuous Markov Processes and Stochastic Equations. *Rendiconti del Circolo Matematico di Palermo*, 4(1), 48–90. <https://doi.org/10.1007/BF02846028>